

Lab 2 Report by Alex Cooper

In the second lab of csc471, the goal is disabling a nag screen which mirrors a pop-up that often occurs from malware on devices. To do this the plan is to use ollydbg to look at the assembly of the executable file "lab2.exe" and skip the undesirable sections. The plan will then shift to locating a hardcoded registration code. The structure of this report will begin by answering all 3 main questions followed by answering the fourth bonus questions. Below the answers will be screen shots to verify I successfully followed the lab steps and ultimately successfully registered.

Question 1: Which CPU register is used to store the return value (1) of the function `rtcMsgBox()`? Why?

Answer 1: The CPU register used to store the return value of a function is **EAX**. According to the lecture slides for chapter 3, "EAX – Accumulator for operands and results data," which means it is intended to hold calculation results and function outputs. In the assembly, we often see `PUSH EAX` before a function call to preserve its previous value, effectively freeing EAX to hold the result of the function. Immediately after the function is executed, EAX contains the return value that the caller can use for further operations such as a comparison or jump.

Question 2: What is the meaning of "`PUSH EBP, MOV EBP, ESP`"?

Answer 2: The above two lines of assembly code are the method by which new stack frames are created. When a function is called inside the executable code the function prologue is used to establish a standard for stack frame management. Pushing the EBP saves the previous base pointer while the movement of EBP gives the function a stable base to manage the addresses for accessing function arguments and local variables. This combination allows for consistent and reliable memory management during execution.

Question 3: Please explain why changing the instruction at 0x402C17 from "`PUSH EBP`" to "`RETN 4`" removes the Nag screen?

Answer 3: In this lab he push EBP and mov EBP, ESP take up 3 bytes worth of space and is critical to creation of the stack frame within the functions prologue. By replacing the push

to a ret we essentially skip the setup of the stack frame. The 4 then also replaces the subsequent mov instructions, in this case the ret 4 is the same size as the stack frame creation. Since the stack frame was never created, the function that establishes the nag screen is unable to execute its body. Instead, it returns as if the nag screen was successful without ever having displayed the nag screen.

Question 4 (BONUS): What is the hardcoded registration code found near the __vbaStrCmp function call?

Answer 4(BONUS): The hard coded registration code is “I’m lena151”.

00401008	.-FF25 18514000	JMP	DWORD	PTR	DS:[<&MSUBUM50._Cisin>]	MSUBUM50._Cisin
0040100E	.-FF25 38514000	JMP	DWORD	PTR	DS:[<&MSUBUM50._Cisqrt>]	MSUBUM50._Cisqrt
00401014	.-FF25 7C514000	JMP	DWORD	PTR	DS:[<&MSUBUM50._Citan>]	MSUBUM50._Citan
0040101A	.-FF25 78514000	JMP	DWORD	PTR	DS:[<&MSUBUM50._allmul>]	MSUBUM50._allmul
00401020	.-FF25 E8504000	JMP	DWORD	PTR	DS:[<&MSUBUM50._vbaFreeU>]	MSUBUM50._vbaFreeU
00401026	.-FF25 2C514000	JMP	DWORD	PTR	DS:[<&MSUBUM50._vbaObjUa>]	MSUBUM50._vbaObjUa
0040102C	.-FF25 04514000	JMP	DWORD	PTR	DS:[<&MSUBUM50._vbaLateH>]	MSUBUM50._vbaLateH
00401032	.-FF25 6C514000	JMP	DWORD	PTR	DS:[<&MSUBUM50._vbaVarDup>]	MSUBUM50._vbaVarDup
00401038	.-FF25 F0504000	JMP	DWORD	PTR	DS:[<&MSUBUM50._vbaEnd>]	MSUBUM50._vbaEnd
0040103E	.-FF25 28514000	JMP	DWORD	PTR	DS:[<&MSUBUM50._vbaVarTs>]	MSUBUM50._vbaVarTs
00401044	.-FF25 EC504000	JMP	DWORD	PTR	DS:[<&MSUBUM50._vbaFreeU>]	MSUBUM50._vbaFreeU
0040104A	.-FF25 68514000	JMP	DWORD	PTR	DS:[<&MSUBUM50._vbaI4Var>]	MSUBUM50._vbaI4Var
00401050	.-FF25 0C514000	JMP	DWORD	PTR	DS:[<&MSUBUM50._#595>]	HERE IS THE CALL
00401056	.-FF25 E4504000	JMP	DWORD	PTR	DS:[<&MSUBUM50._vbaVarMov>]	MSUBUM50._vbaVarMove
0040105C	.-FF25 70514000	JMP	DWORD	PTR	DS:[<&MSUBUM50._vbaVarCop>]	MSUBUM50._vbaVarCopy
00401062	.-FF25 88514000	JMP	DWORD	PTR	DS:[<&MSUBUM50._vbaFreeOb>]	MSUBUM50._vbaFreeObj
00401068	.-FF25 84514000	JMP	DWORD	PTR	DS:[<&MSUBUM50._vbaFreeStr>]	MSUBUM50._vbaFreeStr
0040106E	.-FF25 FC504000	JMP	DWORD	PTR	DS:[<&MSUBUM50._vbaHresu>]	MSUBUM50._vbaHresultCheckObj
00401074	.-FF25 08514000	JMP	DWORD	PTR	DS:[<&MSUBUM50._vbaObjSe>]	MSUBUM50._vbaObjSet
0040107A	.-FF25 24514000	JMP	DWORD	PTR	DS:[<&MSUBUM50._vbaStrCmp>]	MSUBUM50._vbaStrCmp
00401080	.-FF25 3C514000	JMP	DWORD	PTR	DS:[<&MSUBUM50.EVENT_SINK>]	MSUBUM50.EVENT_SINK_QueryInterface
00401086	.-FF25 20514000	JMP	DWORD	PTR	DS:[<&MSUBUM50.EVENT_SINK>]	MSUBUM50.EVENT_SINK_AddRef
0040108C	.-FF25 34514000	JMP	DWORD	PTR	DS:[<&MSUBUM50.EVENT_SINK>]	MSUBUM50.EVENT_SINK_Release
00401092	.-FF25 64514000	JMP	DWORD	PTR	DS:[<&MSUBUM50._#100>]	MSUBUM50.ThunRTMain
00401168	.\$ 68 B8184000	PUSH	DWORD		004018B8	
0040116D	.\$ E8 F0FFFFFF	CALL	<JMP.&MSUBUM50._#100>			
00401172	.\$ 0000	ADD	BYTE	PTR	DS:[EAX],AL	
00401174	.\$ 0000	ADD	BYTE	PTR	DS:[EAX],AL	
00401176	.\$ 0000	ADD	BYTE	PTR	DS:[EAX],AL	

00402C07	.\$ 58	POP	EAX			
00402C08	.\$ 894D 94	MOV	DWORD	PTR	SS:[EBP-6C],ECX	
00402C0B	.\$ 8945 8C	MOV	DWORD	PTR	SS:[EBP-74],EAX	
00402C0E	.\$ 8945 9C	MOV	DWORD	PTR	SS:[EBP-64],EAX	
00402C11	.\$ 8D45 8C	LEA	EAX	DWORD	PTR	SS:[EBP-74]
00402C14	.\$ 894D A4	MOV	DWORD	PTR	SS:[EBP-5C],ECX	
00402C17	.\$ 58	PUSH	EAX			
00402C18	.\$ 8D45 9C	LEA	EAX	DWORD	PTR	SS:[EBP-64]
00402C1B	.\$ 58	PUSH	EAX			
00402C1C	.\$ 8D45 CC	LEA	EAX	DWORD	PTR	SS:[EBP-34]
00402C1F	.\$ 58	PUSH	EAX			
00402C20	.\$ 8D45 DC	LEA	EAX	DWORD	PTR	SS:[EBP-24]
00402C23	.\$ 58	PUSH	EAX			
00402C24	.\$ E8 21E4FFFF	CALL	<JMP.&MSUBUM50._vbaI4Var>			
00402C27	.\$ 58	PUSH	EAX			
00402C28	.\$ 8D45 AC	LEA	EAX	DWORD	PTR	SS:[EBP-54]
00402C2B	.\$ 58	PUSH	EAX			
00402C2C	.\$ 83C4 14	ADD	ESP	14		
00402D01	.\$ 90	NOP				
00402D02	.\$ 90	NOP				
00402D03	.\$ 8D95 SCFFFFFF	LEA	EDX	DWORD	PTR	SS:[EBP-A4]
00402D06	.\$ 8D4D BC	LEA	ECX	DWORD	PTR	SS:[EBP-44]
00402D09	.\$ 8985 64FFFFFF	MOV	DWORD	PTR	SS:[EBP-9C],EAX	
00402D12	.\$ 89BD SCFFFFFF	MOV	DWORD	PTR	SS:[EBP-A4],EDI	
00402D15	.\$ E8 09E4FFFF	CALL	<JMP.&MSUBUM50._vbaVarMove>			
00402D18	.\$ 8D45 8C	LEA	EAX	DWORD	PTR	SS:[EBP-74]
00402D1B	.\$ 58	PUSH	EAX			
00402D1C	.\$ 8D45 9C	LEA	EAX	DWORD	PTR	SS:[EBP-64]
00402D1F	.\$ 58	PUSH	EAX			

```
0040288D . 8D45 A4 LEA EAX,DWORD PTR SS:[EBP-5C]
00402890 . 50 PUSH EAX
00402891 . E8 AEE8FFFF CALL <JMP.&MSUBUM50.__vbaObjSet>
00402896 . 8BF8 MOV EDI,EAX
00402898 . 8D4D A8 LEA ECX,DWORD PTR SS:[EBP-58]
0040289B . 51 PUSH ECX
0040289C . 57 PUSH EDI
0040289D . 8B07 MOV EAX,DWORD PTR DS:[EDI]
0040289F . FF90 A0000000 CALL DWORD PTR DS:[EAX+A0]
004028A5 . 3BC6 CMP EAX,ESI
004028A7 . 7D 11 JGE SHORT lab2.004028BA
004028A9 . 68 A0000000 PUSH 0A0
004028AE . 68 F41D4000 PUSH lab2.00401DF4
004028B3 . 57 PUSH EDI
004028B4 . 50 PUSH EAX
004028B5 . E8 84E8FFFF CALL <JMP.&MSUBUM50.__vbaHresultCheckObj>
004028BA > FF75 A8 PUSH DWORD PTR SS:[EBP-58]
004028BD . 68 DC1D4000 PUSH lab2.00401DDC
004028C2 . E8 83E8FFFF CALL <JMP.&MSUBUM50.__vbaStrCmp>
004028C7 . 8BF8 MOV EDI,EAX
004028C9 . 8D4D A8 LEA ECX,DWORD PTR SS:[EBP-58]
004028CC . F7DF NEG EDI
004028CE . 1BFF SBB EDI,EDI
004028D0 . 47 INC EDI
004028D1 . F7DF NEG EDI
004028D3 . E8 60E8FFFF CALL <JMP.&MSUBUM50.__vbaFreeStr>
004028D8 . 8D4D A4 LEA ECX,DWORD PTR SS:[EBP-5C]
004028DB . E8 52E8FFFF CALL <JMP.&MSUBUM50.__vbaFreeObj>
004028E0 . 66:3BFE CMP DI,SI
004028E3 . 65:52 52000000 IF LEBQ 004028DC
00401DDC=lab2.00401DDC (UNICODE "I'm lena151")
```

